Übungsblatt 07: Routing, Topologieänderung und Wireshark

Rechnernetze und Netzwerkprogrammierung, WS 2026/27
Raphael Bleier, Devis, Fabio, Gruppe 1
Erklärung

Gemessen am 30.10.2026 mit Mininet 2.3.0 und Python 3.13.3. `xterm` fehlt hier, deshalb liefen `ip`, `ping` und `tracepath` in den Namespaces. Der Knoten heißt in der Vorlage `client`, in der Angabe `client1`. Nichts wurde auf Moodle hochgeladen.

Erklärung

Die Vorlage setzt Adressen in `start_lab.py`, nicht in `topology.py`. r3 kommt in der Vorlage nicht vor. Die neuen Adressen stehen deshalb in der `start_lab.py` dieses Ordners. Die Zahlen unten sind der Lauf vom 30.10.2026, nicht gerundet aus dem Bauch.

Aufgabe 7.1

Vollständigen Pfad eines IP-Pakets von client1 zum Server bestimmen. Für jeden Hop Interface und Next Hop dokumentieren. Welche Routingtabelle wird auf jedem Router verwendet? Welche Entscheidung trifft r1, welche r2? Warum braucht der Client keine Route für jedes Zielgerät?

Jeder Router hat seine eigene Linux-Tabelle im eigenen Namespace. Eine gemeinsame Labortabelle gibt es nicht. Auf r1, r2 und r3 war `net.ipv4.ip_forward` gleich 1.

`ip route get 10.0.2.2` auf dem Client:

```
10.0.2.2 via 10.0.1.1 dev client-eth0 src 10.0.1.2
```

10.0.2.2 liegt nicht in 10.0.1.0/24. Die Connected-Route greift nicht. Die Default-Route schickt das Paket an 10.0.1.1, Ausgang client-eth0.

Erklärung

Eine Default-Route ist die Sammelroute für alles, was keine längere passende Route hat. Der Client muss die Serveradresse nicht kennen. Er muss nur wissen, dass sie nicht lokal ist. Deshalb reicht ein Eintrag für alle fremden Ziele.

Auf r1:

```
10.0.2.2 via 10.0.12.2 dev r1-eth1 src 10.0.12.1
```

r1 nimmt das Paket auf r1-eth0 an. Die längste passende Route ist `10.0.2.0/24 via 10.0.12.2 dev r1-eth1`. Next Hop ist r2, Ausgang r1-eth1. r1 stellt nicht lokal zu.

Auf r2:

```
10.0.2.2 dev r2-eth1 src 10.0.2.1
```

Kein `via`. 10.0.2.2 liegt in 10.0.2.0/24, das an r2-eth1 hängt. r2 löst die MAC-Adresse per ARP und stellt an server-eth0 zu.

Rückweg, gemessen mit `ip route get 10.0.1.2`: Server über `default via 10.0.2.1 dev server-eth0`, r2 über `10.0.1.0/24 via 10.0.12.1 dev r2-eth0`, r1 direkt über r1-eth0.

Ping client nach 10.0.2.2, fünf Pakete: 5 empfangen, 0 % Verlust, TTL 62. Linux startet bei 64. Zwei Router haben die TTL um 2 gesenkt.

```
tracepath -n 10.0.2.2
 1: 10.0.1.1
 2: 10.0.12.2
 3: 10.0.2.2 reached
    Resume: pmtu 1500 hops 3 back 3
```

Erklärung

TTL 62 und drei tracepath-Zeilen sagen dasselbe: Client, dann r1, dann r2, dann Server. 10.0.12.2 ist r2-eth0, also der Next Hop, den r1 gewählt hat. Wäre die Route auf r1 falsch, käme der zweite Hop woanders her oder der Ping käme mit Unreachable zurück.

client-eth0 → r1-eth0 → r1-eth1 → r2-eth0 → r2-eth1 → server-eth0

Aufgabe 7.2

topology.py um r3 und einen alternativen Weg zwischen r1 und dem Server erweitern. Neue Verbindungen für die Router-zu-Router-Kommunikation. Statische Routen so setzen, dass client1 den Server erreicht. Jeweils nur einen Pfad aktiv testen. Neue Netze, Routingtabellen von r1 und r3, ping und tracepath dokumentieren.

Der Satz zu den neuen Verbindungen bricht in der Angabe ab. Ich lese ihn als neue Punkt-zu-Punkt-Netze, so wie die vorhandene /30 zwischen r1 und r2. Die Skizze kann man auch als direkten Anschluss von r3 an den Server lesen. Das wäre keine Router-zu-Router-Strecke. r3 hängt deshalb zwischen r1 und r2. Das Server-LAN 10.0.2.0/24 bleibt an r2.

Erklärung

Eine /30 hat vier Adressen: Netz, zwei Hosts, Broadcast. Genau zwei Router, keine Reserve für einen Endhost. Dieselbe Wahl steht schon auf der Strecke r1–r2 in der Vorlage. 10.0.13.0/30 und 10.0.23.0/30 überlappen keines der vorhandenen Netze.

Link	Interface	Adresse	Netz
r1–r3	r1-eth2	10.0.13.1/30	10.0.13.0/30
r1–r3	r3-eth0	10.0.13.2/30	10.0.13.0/30
r3–r2	r3-eth1	10.0.23.1/30	10.0.23.0/30
r3–r2	r2-eth2	10.0.23.2/30	10.0.23.0/30

Pfad A, nur diese Fernrouten:

```
r1: 10.0.2.0/24 via 10.0.12.2 dev r1-eth1
r2: 10.0.1.0/24 via 10.0.12.1 dev r2-eth0
```

Pfad B, die beiden Einträge werden vorher gelöscht:

```
r1: 10.0.2.0/24 via 10.0.13.2 dev r1-eth2
r2: 10.0.1.0/24 via 10.0.23.1 dev r2-eth2
r3: 10.0.2.0/24 via 10.0.23.2 dev r3-eth1
r3: 10.0.1.0/24 via 10.0.13.1 dev r3-eth0
```

Erklärung

Zwei statische Routen zum selben Präfix wären ECMP oder eine Metrikwahl. Die Angabe will nur einen aktiven Pfad. Deshalb steht in r1 immer genau ein Eintrag für 10.0.2.0/24. `ip route del` vor `replace` räumt einen alten Eintrag weg, falls das Präfix schon da ist.

Auf Pfad B, gemessen:

```
r1: 10.0.2.2 via 10.0.13.2 dev r1-eth2 src 10.0.13.1
r3: 10.0.2.2 via 10.0.23.2 dev r3-eth1 src 10.0.23.1
r2: 10.0.2.2 dev r2-eth1 src 10.0.2.1
```

Ping: 5 von 5, 0 % Verlust, TTL 61. Drei Router. `tracepath -n` nannte 10.0.1.1, dann 10.0.13.2. Hop 3 antwortete nicht, Hop 4 war 10.0.2.2, `hops 4 back 4`.

Erklärung

Hop 3 fehlt in der Ausgabe, die TTL nicht. 61 ist 64 minus 3. r2 hat den Probe nicht beantwortet, das Paket ist aber angekommen. Ein einzelner fehlender Hop ist kein Beweis für einen Abbruch, wenn Ping und TTL den dritten Router zeigen.

$$\boxed{\text{Pfad B: client} \rightarrow \text{r1} \rightarrow \text{r3} \rightarrow \text{r2} \rightarrow \text{server, TTL 61}}$$

Aufgabe 7.3

Den beiden Wegen unterschiedliche Linkverzögerungen geben. Beispiel delayA = 5 ms, delayB = 30 ms. RTT mit ping messen und vergleichen. Warum ist die beobachtete RTT nicht einfach die Summe der angegebenen Linkwerte?

Verzögerung pro Richtung, mit `tc qdisc show` geprüft: client–r1 und r2–server je 10 ms (Vorlage), r1–r2 5 ms, r1–r3 und r3–r2 je 30 ms. TCLink setzt denselben Wert in beide Richtungen.

$$\begin{aligned}
t_A &= 10\,\text{ms} + 5\,\text{ms} + 10\,\text{ms} && \text{drei Links, eine Richtung} \\
t_A &= 25\,\text{ms} && 10 + 5 + 10 = 25 \\
\text{RTT}_A &= 2 \cdot 25\,\text{ms} && \text{netem gilt pro Richtung} \\
\text{RTT}_A &= 50\,\text{ms} && 2 \cdot 25 = 50
\end{aligned}$$

$$\begin{aligned}
t_B &= 10\,\text{ms} + 30\,\text{ms} + 30\,\text{ms} + 10\,\text{ms} && \text{vier Links, eine Richtung} \\
t_B &= 80\,\text{ms} && 10 + 30 + 30 + 10 = 80 \\
\text{RTT}_B &= 2 \cdot 80\,\text{ms} && \text{netem gilt pro Richtung} \\
\text{RTT}_B &= 160\,\text{ms} && 2 \cdot 80 = 160
\end{aligned}$$

Gemessen, fünf spätere Pings, Pfad A: min/avg/max = 50,057/50,071/50,087 ms, TTL 62, 0 % Verlust. Der einzelne Ping davor: 100,206 ms.

Pfad B: 160,081/160,143/160,186 ms, TTL 61, 0 % Verlust. Der einzelne Ping davor: 280,512 ms.

Erklärung

Die Angabe nennt 5 ms und 30 ms. Wer nur diese beiden Zahlen addiert, landet bei 35 ms und trifft keine der Messungen. Ping zählt Hin- und Rückweg. Die gemeinsamen 10-ms-Links stecken in beiden Pfaden. Pfad B hat zwei Links mit 30 ms. Das erste Paket wartet zusätzlich auf ARP, das in der Linksumme nicht vorkommt. Die späteren Pakete liegen nah an 50 ms bzw. 160 ms, das erste nicht.

spätere RTT: Pfad A 50,071 ms, Pfad B 160,143 ms

Aufgabe 7.4

Absichtlich eine statische Route entfernen, Beispiel Route von r3 zum Server. Ursache mit ping, traceroute und den besprochenen Tools finden. Dokumentieren, welche Tests zuerst kommen, was ip addr und ip route liefern, wann ping, traceroute und Wireshark den Hinweis geben. Fehler, Diagnose und Korrektur festhalten.

Auf Pfad B nur diese Route gelöscht:

```
r3: ip route del 10.0.2.0/24
```

Die Route von r3 nach 10.0.1.0/24 blieb.

Zuerst die Nachbarn. client nach 10.0.1.1: 1 von 1, 20,022 ms, TTL 64. r1 nach 10.0.13.2: 1 von 1, 60,129 ms, TTL 64. Das ist zweimal 30 ms auf diesem einen Link. Die Strecke bis r3 steht.

`ip -br addr` auf r3: r3-eth0 `UP` mit 10.0.13.2/30, r3-eth1 `UP` mit 10.0.23.1/30. Die Interfaces sind nicht das Problem.

`ip route` auf r3 danach:

```
10.0.1.0/24 via 10.0.13.1 dev r3-eth0
10.0.13.0/30 dev r3-eth0 proto kernel scope link src 10.0.13.2
10.0.23.0/30 dev r3-eth1 proto kernel scope link src 10.0.23.1
```

`10.0.2.0/24` fehlt. `ip route get 10.0.2.2` antwortete `Network is unreachable`.

Dann der Ping vom Client zum Server: `From 10.0.13.2 Destination Net Unreachable`, 3 gesendet, 0 empfangen, 100 % Verlust. 10.0.13.2 ist r3-eth0.

Erklärung

`ip addr` sagt, ob das Interface existiert und eine Adresse hat. `ip route` sagt, ob der Router das Zielnetz kennt. Ping zum Nachbarn trennt “Link tot” von “Route fehlt”. Ping zum Server nennt den Absender der Fehlermeldung. Hier ist das 10.0.13.2, also r3, nicht der Client und nicht der Server.

`tracepath -n 10.0.2.2` im Fehlerfall: Hop 1 war 10.0.1.1, danach `no reply` bis Hop 30. Der Absender der Unreachable-Meldung taucht dort nicht als Hop auf.

Mitschnitt auf r3-eth0, gelesen mit tshark, kein Wireshark-Fenster (`xterm` fehlt):

```
10.0.1.2 -> 10.0.2.2 ICMP Typ 8 Code 0 Echo request TTL 63
10.0.13.2 -> 10.0.1.2 ICMP Typ 3 Code 0 Destination unreachable
                    Network unreachable
```

TTL 63 heißt: das Echo hat r1 schon passiert und ist auf r3 angekommen. Typ 3 Code 0 ist Network unreachable. r3 hat nicht weitergeleitet.

Erklärung

Wireshark wird entscheidend, wenn ping nur “timeout” sagt oder tracepath keinen Absender nennt. Hier hat ping den Absender schon genannt. Der Mitschnitt bestätigt ihn am Interface r3-eth0 und zeigt, dass das Paket ankam und nicht irgendwo davor verloren ging. `ip addr` kann diesen Unterschied nicht zeigen.

Korrektur:

```
r3: ip route replace 10.0.2.0/24 via 10.0.23.2 dev r3-eth1
```

Danach Ping: 3 von 3, TTL 61, RTT 160,133 bis 160,686 ms.

Fehler: r3 ohne 10.0.2.0/24, ICMP Typ 3 Code 0 von 10.0.13.2

Aufgabe 7.5

CSMA beschreiben. Verhalten bei freiem und belegtem Kanal, und warum es trotzdem Kollisionen gibt. Unterschied zu CSMA/CD, Reaktion bei erkannter Kollision. In welchen Netzen CSMA/CD oder CSMA/CA?

Die Folien der VO bis Kapitel 3 enthalten diesen Abschnitt nicht. Die VO nennt Kurose/Ross. Die Antworten folgen dieser Darstellung und IEEE 802.3 bzw. 802.11, nicht einer Folie, die hier noch nicht liegt.

CSMA (Carrier Sense Multiple Access): Der Knoten hört den gemeinsamen Kanal ab, bevor er sendet. Ist der Kanal frei, sendet er. Ist er belegt, wartet er und sendet nicht sofort. Kollisionen bleiben möglich, weil das Signal Laufzeit hat. Zwei Knoten können beide Stille hören und beide anfangen, bevor das Signal des anderen ankommt.

CSMA schickt den Rahmen zu Ende, auch wenn unterwegs eine Kollision entsteht. CSMA/CD hört während des Sendens mit. Weicht das Empfangene vom Gesendeten ab, bricht die Station den Rahmen ab, sendet ein Jam-Signal und wartet eine zufällige Zeit (exponential backoff), bevor sie es erneut versucht. Das Jam-Signal soll andere Empfänger den beschädigten Rahmen verwerfen lassen.

CSMA/CD wird im klassischen gemeinsamen Ethernet eingesetzt, IEEE 802.3, Koax und Hubs, halbduplex. Auf einer geschalteten Vollduplex-Strecke gibt es diesen Wettbewerb nicht, dort läuft CSMA/CD nicht. CSMA/CA wird in IEEE 802.11 eingesetzt. Eine Funkstation kann beim Senden schlecht mithören, das eigene Signal übertönt den Kanal. Sie wartet, bis der Kanal frei ist, sendet den Rahmen ganz und erwartet eine Bestätigung. Bleibt die aus, nimmt sie eine Kollision an und wartet mit exponential backoff. RTS/CTS ist eine zusätzliche Absprache vor dem Senden, kein Pflichtteil jeder Übertragung.

Erklärung

Die Frage trennt drei Dinge. CSMA ist das Abhören vor dem Senden. CD ist das Abbrechen während des Sendens, das geht auf einem gemeinsamen Kupfersegment. CA ist das Vermeiden vor dem Senden, das braucht WLAN, weil Mithören beim Senden dort schlecht geht. Wer CD und CA vertauscht, nennt beim Ethernet die falsche Reaktion: WLAN bricht den Rahmen nicht mitten im Senden ab.

CSMA/CD: klassisches Ethernet (802.3). CSMA/CA: WLAN (802.11).